GO LANGUAGE
INTERNAL EXAM STUDY GUIDE

Experiments 1–12 • Simple Go code • Explanations • Examples and viva

Excluded: Experiments 13 and 14, as requested.

Based on your uploaded GO_LAB.pdf. Simplified code may differ from the exact format your lecturer expects.

Experiment 1 — GCD and LCM

Lab record: 2–4 **Aim:** Find GCD and LCM of two positive, nonzero integers.

```
package main
import "fmt"
func main() {
    var a, b int
    fmt.Print("Enter two numbers: ")
    fmt.Scan(&a, &b)
    x, y := a, b
    for y != 0 {
        x, y = y, x%y
    }
    gcd := x
    lcm := (a * b) / gcd
    fmt.Println("GCD =", gcd)
    fmt.Println("LCM =", lcm)
}
```

How it works: The loop repeatedly calculates remainder using %. When y becomes zero, x is the GCD. LCM = (a × b) / GCD.

Examples: 12 and 18 → GCD 6, LCM 36. 8 and 12 → GCD 4, LCM 24.

Viva: GCD × LCM = a × b.

Experiment 2 — Number Pyramid

Lab record: 5–7 **Aim:** Print a right-aligned number pyramid.

```
package main
import "fmt"
func main() {
    var n int
    fmt.Print("Enter number of rows: ")
    fmt.Scan(&n)
    for i := 1; i <= n; i++ {
        for j := 1; j <= n-i; j++ { fmt.Print(" ") }
        for j := 1; j <= i; j++ { fmt.Print(j, " ") }
        fmt.Println()
    }
}
```

How it works: Outer loop controls rows. First inner loop prints spaces; second prints numbers from 1 to the row number.

Examples: 3 rows: 1 / 1 2 / 1 2 3. 5 rows end with 1 2 3 4 5.

Viva: Nested loops handle spaces and numbers.

Experiment 3 — Student Struct

Lab record: 8–10 **Aim:** Store and display student details.

Note: The lab aim says import a struct from another package. This simpler single-file version demonstrates structs but not a separate package.

```
package main
import "fmt"
type Student struct {
    ID int
    Name string
    Email string
}
func main() {
    s := Student{ID: 10, Name: "Maverick", Email: "maverick@gmail.com"}
    fmt.Println("ID:", s.ID)
    fmt.Println("Name:", s.Name)
    fmt.Println("Email:", s.Email)
}
```

How it works: A struct groups related fields. Use the dot operator, such as s.Name, to access a field.

Examples: Student{ID: 1, Name: "Ravi", Email: "ravi@gmail.com"}. Student{ID: 2, Name: "Sai", Email: "sai@gmail.com"}.

Viva: A struct groups fields of different data types.

Experiment 4 — Standard Deviation

Lab record: 11–13 **Aim:** Calculate population standard deviation.

```
package main
import ("fmt"; "math")
func main() {
    var n int
    fmt.Print("Enter size: ")
    fmt.Scan(&n)
    if n <= 0 { fmt.Println("Invalid size"); return }
    nums := make([]float64, n)
    var sum float64
    for i := 0; i < n; i++ {
        fmt.Scan(&nums[i])
        sum += nums[i]
    }
    mean := sum / float64(n)
    var total float64
    for i := 0; i < n; i++ {
        diff := nums[i] - mean
        total += diff * diff
    }
    sd := math.Sqrt(total / float64(n))
    fmt.Println("Standard deviation:", sd)
}
```

How it works: Find the mean. Square each number's difference from the mean, add the squares, divide by n, then take the square root. math.Sqrt calculates square root.

Examples: 10, 10, 10 → 0. 2, 4, 6 → about 1.633.

Viva: This uses population standard deviation because it divides by n.

Experiment 5 — Floyd's Triangle

Lab record: 14–16 **Aim:** Print consecutive numbers in a triangle.

```
package main
import "fmt"
func main() {
    var rows int
    fmt.Print("Enter number of rows: ")
    fmt.Scan(&rows)
    num := 1
    for i := 1; i <= rows; i++ {
        for j := 1; j <= i; j++ {
            fmt.Print(num, " ")
            num++
        }
        fmt.Println()
    }
}
```

How it works: Outer loop controls rows. Inner loop prints i numbers. num++ increases the number after every print.

Examples: 2 rows: 1 / 2 3. 3 rows: 1 / 2 3 / 4 5 6.

Viva: num is outside the loops so counting continues across rows.

Experiment 6 — Add Two Strings

Lab record: 17–18 **Aim:** Join two strings.

```
package main
import "fmt"
func main() {
    var s1, s2 string
    fmt.Print("Enter first string: ")
    fmt.Scan(&s1)
    fmt.Print("Enter second string: ")
    fmt.Scan(&s2)
    result := s1 + " " + s2
    fmt.Println("Addition of two strings:", result)
}
```

How it works: The + operator joins strings; the quoted space separates them. fmt.Scan reads only up to whitespace.

Examples: Hello + World → Hello World. Good + Morning → Good Morning.

Viva: String concatenation means joining strings.

Experiment 7 — Palindrome

Lab record: 19–21 **Aim:** Check whether a string reads the same forward and backward.

Note: This simple version is for basic English ASCII words.

```
package main
import "fmt"
func main() {
    var str string
    fmt.Print("Enter a string: ")
    fmt.Scan(&str)
    reverse := ""
    for i := len(str)-1; i >= 0; i-- {
        reverse += string(str[i])
    }
    if str == reverse {
        fmt.Println("The given string is a palindrome")
    } else {
        fmt.Println("The given string is not a palindrome")
    }
}
```

How it works: Build a reversed string from the last index to the first, then compare it with the original.

Examples: madam → palindrome. hello → not a palindrome.

Viva: A palindrome stays the same when reversed.

Experiment 8 — Contact Details

Lab record: 22–24 **Aim:** Read and display contact details using a struct.

Note: This is a console program, not a browser-based form.

```
package main
import "fmt"
type Contact struct {
    Name string
    Age int
    Phone string
    Email string
}
func main() {
    var c Contact
    fmt.Print("Enter name: ")
    fmt.Scan(&c.Name)
    fmt.Print("Enter age: ")
    fmt.Scan(&c.Age)
    fmt.Print("Enter phone: ")
    fmt.Scan(&c.Phone)
    fmt.Print("Enter email: ")
    fmt.Scan(&c.Email)
    fmt.Println("\nContact Details")
    fmt.Println("Name:", c.Name)
    fmt.Println("Age:", c.Age)
    fmt.Println("Phone:", c.Phone)
    fmt.Println("Email:", c.Email)
}
```

How it works: Contact groups four fields. Read values into each field and display them with the dot operator.

Examples: Ravi, 20, 9876543210, ravi@gmail.com. Sai, 21, 9123456780, sai@gmail.com.

Viva: Phone is a string because it may contain leading zeros or a country code.

Experiment 9 — Average Using a Slice

Lab record: 25–26 **Aim:** Calculate the average of entered numbers.

```
package main
import "fmt"
func main() {
    var n int
    fmt.Print("Enter N: ")
    fmt.Scan(&n)
    if n <= 0 { fmt.Println("Invalid size"); return }
    arr := make([]int, n)
    sum := 0
    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
        sum += arr[i]
    }
    avg := float64(sum) / float64(n)
    fmt.Println("Average:", avg)
}
```

How it works: Store each number in a slice and add it to sum. Divide sum by n. float64 allows decimal averages.

Examples: 10, 20, 30, 40, 50 → 30. 2, 3, 4 → 3.

Viva: Average = sum of elements / number of elements.

Experiment 10 — Remove Duplicates

Lab record: 27–29 **Aim:** Keep only the first occurrence of each integer.

```
package main
import "fmt"
func main() {
    var n int
    fmt.Print("Enter size: ")
    fmt.Scan(&n)
    if n <= 0 { fmt.Println("Invalid size"); return }
    arr := make([]int, n)
    seen := make(map[int]bool)
    result := []int{}
    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
        if !seen[arr[i]] {
            seen[arr[i]] = true
            result = append(result, arr[i])
        }
    }
    fmt.Println("Result:", result)
}
```

How it works: The map tracks numbers already seen. If a number is new, mark it true and append it to result. Repeated numbers are skipped.

Examples: [10, 20, 30, 20, 10] → [10 20 30]. [5, 5, 5] → [5].

Viva: A map stores key-value pairs; here values are Boolean.

Experiment 11 — Reverse Sort

Lab record: 30–32 **Aim:** Sort integers descending and strings in reverse alphabetical order.

Note: If your lecturer expects the sorting algorithm itself, practise the nested-loop swapping version from the lab record.

```
package main
import ("fmt"; "sort")
func main() {
    nums := []int{10, 5, 30, 20, 15}
    words := []string{"Apple", "Mango", "Banana", "Orange"}
    sort.Sort(sort.Reverse(sort.IntSlice(nums)))
    sort.Sort(sort.Reverse(sort.StringSlice(words)))
    fmt.Println("Reverse Sorted Integers:", nums)
    fmt.Println("Reverse Sorted Strings:", words)
}
```

How it works: IntSlice and StringSlice make slices sortable. sort.Reverse changes order to descending.

Examples: [10, 5, 30, 20, 15] → [30 20 15 10 5]. Apple, Mango, Banana → Mango, Banana, Apple.

Viva: Ascending is low-to-high; descending is high-to-low.

Experiment 12 — String Functions

Lab record: 33–34 **Aim:** Demonstrate Contains, ContainsAny, Count, and EqualFold.

```
package main
import ("fmt"; "strings")
func main() {
    str := "Hello Golang"
    fmt.Println("Contains:", strings.Contains(str, "Go"))
    fmt.Println("ContainsAny:", strings.ContainsAny(str, "xyz"))
    fmt.Println("Count:", strings.Count(str, "l"))
    fmt.Println("EqualFold:", strings.EqualFold("Golang", "golang"))
}
```

How it works: Contains checks a substring. ContainsAny checks whether any listed character occurs. Count counts occurrences. EqualFold compares without case sensitivity.

Examples: Contains("Hello", "ell") → true. EqualFold("GO", "go") → true.

Viva: These functions are provided by the strings package.

Quick Revision Checklist

1. GCD/LCM: remainder %, loop, formula.
2. Pyramid: nested loops, spaces, rows.
3. Struct: fields and dot operator.
4. Standard deviation: mean, squared differences, math.Sqrt.
5. Floyd's triangle: nested loops, num++.
6. String addition: concatenation using +.
7. Palindrome: reverse and compare.
8. Contact details: struct and input.
9. Average: sum divided by n.
10. Duplicates: map, Boolean, append.
11. Reverse sorting: sort package.
12. String functions: Contains, ContainsAny, Count, EqualFold.

Go basics: package main, import, func main(), fmt.Scan, fmt.Println, for, if, slices, maps, structs, and type conversions.

Reminder: Experiments 13 and 14 are excluded. Code is simplified for learning; follow any specific format your lecturer requires.